

MAB_Clinical_Trial_Assignment

June 21, 2025

1 Group ID: 201

1.0.1 Group Members:

1. JAGADI SHRUTI CHANABASAPPA: 2024AA05181
2. TANAY KASHYAP: 2024AA05991
3. SHIKHAR NIGAM: 2024AA05691
4. KHUSHWANT SINGH RATHORE: 2024AA05490

Course: Deep Reinforcement Learning

Assignment: Lab Assignment 1 — Multi-Armed Bandits (Part 1)

Institution: BITS Pilani WILP

2 Scenario

A pharmaceutical company is conducting clinical trials to evaluate the effectiveness of three antiretroviral drug combinations for treating HIV-positive patients. Due to the ethical and cost constraints of clinical trials, it is critical to identify the most effective treatment regimen using the least number of patients. Each treatment (or “arm”) can lead to different outcomes depending on patient responses. The effectiveness of each treatment is evaluated using a reward function derived from the improvement in patients’ immune system markers and survival status.

Group 201 — DRL Lab Assignment (MAB Part)

This notebook simulates a clinical trial environment using Multi-Armed Bandit (MAB) strategies to maximize successful outcomes for HIV treatments. Various MAB policies (Random, Greedy, -Greedy, UCB) are implemented and evaluated.

2.1 Reward Function

A successful treatment is defined as:

Reward $r = 1$, if (label == 0) and (cd420 > cd40), else $r = 0$

3 Dataset

You will be provided a dataset containing the following fields:

- **Age (*age*):** Patient's age in years at baseline.
- **Weight (*wtkg*):** Continuous feature representing weight in kilograms at baseline.
- **Gender (*gender*):** Binary indicator of gender (0 = Female, 1 = Male).
- **CD4 Counts (*cd40*, *cd420*):** Integer values representing CD4 counts at baseline and 20+/- 5 weeks.
- **Treatment Indicator (*trt*):** Categorical feature indicating the type of treatment received (0 = ZDV only, 1 = ZDV + ddI, 2 = ZDV + Zai, 3 = ddI only).
- **Censoring Indicator (*label*):** Binary indicator (1 = failure, 0 = censoring) denoting patient status.

Link for accessing dataset: https://drive.google.com/file/d/1LYfIrJ4VEEGeyOsSt_qoLk7FaAv5Jfx-/view?usp=sharing

3.0.1 Treatment Arms:

- **0:** ZDV only
- **1:** ZDV + ddI
- **2:** ZDV + Zai
- **3:** ddI only

3.0.2 Step 1: Load the Clinical Trial Dataset

We begin by importing the dataset containing information about patient treatments, CD4 counts, and outcomes. This will be used to simulate our MAB environment.

```
[9]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import defaultdict

# Load dataset
data = pd.read_csv("Clinical_Trial.csv")
data = data.dropna()

print("Sample dataset:")
data.head()
```

Sample dataset:

```
[9]:
```

	age	wtkg	gender	cd40	cd420	trt	label
0	48	89.8128	0	422	477	2	0
1	61	49.4424	0	162	218	3	1
2	45	88.4520	1	326	274	3	0
3	47	85.2768	1	287	394	3	0
4	43	66.6792	1	504	353	0	0

3.0.3 Step 2: Design a clinical trial environment

Here, we define a reward function based on patient survival and improvement in CD4 count as per the assignment specification.

```
[10]: class ClinicalTrialEnvironment:
    def __init__(self, data):
        self.data = data.sample(frac=1).reset_index(drop=True)  # Shuffle data
        self.index = 0

    def reset(self):
        self.index = 0

    def step(self, action):
        if self.index >= len(self.data):
            raise IndexError("No more data to sample")

        row = self.data.iloc[self.index]
        self.index += 1

        reward = 1 if row['label'] == 0 and row['cd420'] > row['cd40'] and
        row['trt'] == action else 0
        return reward
```

3.0.4 Step 3: Random Policy Simulation

```
[11]: def run_random_policy(env, iterations=1000):
    rewards = []
    arm_counts = [0]*4

    for _ in range(iterations):
        action = np.random.choice(4)
        reward = env.step(action)
        rewards.append(reward)
        arm_counts[action] += 1

    return rewards, arm_counts

env = ClinicalTrialEnvironment(data)
random_rewards, random_counts = run_random_policy(env)
print("Random Policy: Total Reward =", sum(random_rewards))
```

Random Policy: Total Reward = 101

3.0.5 Step 4: Greedy Policy Simulation

```
[12]: def run_greedy_policy(env, iterations=1000):
    total_rewards = [0]*4
    counts = [1e-5]*4 # avoid division by zero
    rewards = []
    arm_counts = [0]*4

    for _ in range(iterations):
        avg_rewards = [total_rewards[i]/counts[i] for i in range(4)]
        action = np.argmax(avg_rewards)
        reward = env.step(action)
        total_rewards[action] += reward
        counts[action] += 1
        rewards.append(reward)
        arm_counts[action] += 1

    return rewards, arm_counts

env.reset()
greedy_rewards, greedy_counts = run_greedy_policy(env)
print("Greedy Policy: Total Reward =", sum(greedy_rewards))
```

Greedy Policy: Total Reward = 77

3.0.6 Step 5: Epsilon Greedy Policy Simulation

```
[13]: def run_epsilon_greedy(env, iterations=1000, epsilon=0.1):
    total_rewards = [0]*4
    counts = [1e-5]*4
    rewards = []
    arm_counts = [0]*4

    for _ in range(iterations):
        if np.random.rand() < epsilon:
            action = np.random.choice(4)
        else:
            avg_rewards = [total_rewards[i]/counts[i] for i in range(4)]
            action = np.argmax(avg_rewards)
        reward = env.step(action)
        total_rewards[action] += reward
        counts[action] += 1
        rewards.append(reward)
        arm_counts[action] += 1

    return rewards, arm_counts

epsilons = [0.1, 0.2, 0.5]
```

```

epsilon_results = {}

for eps in epsilons:
    env.reset()
    rewards, counts = run_epsilon_greedy(env, epsilon=eps)
    epsilon_results[eps] = (rewards, counts)
    print(f"Epsilon = {eps}: Total Reward = {sum(rewards)}")

```

Epsilon = 0.1: Total Reward = 126

Epsilon = 0.2: Total Reward = 125

Epsilon = 0.5: Total Reward = 115

3.0.7 Step 6: UCB Policy Simulation

```

[14]: def run_ucb_policy(env, iterations=1000):
    total_rewards = [0]*4
    counts = [0]*4
    rewards = []
    arm_counts = [0]*4

    for t in range(1, iterations + 1):
        ucb_values = [float('inf') if counts[i] == 0 else
                      (total_rewards[i]/counts[i]) + np.sqrt((2*np.log(t))/
↪counts[i])] for i in range(4)]
        action = np.argmax(ucb_values)
        reward = env.step(action)
        total_rewards[action] += reward
        counts[action] += 1
        rewards.append(reward)
        arm_counts[action] += 1

    return rewards, arm_counts

env.reset()
ucb_rewards, ucb_counts = run_ucb_policy(env)
print("UCB Policy: Total Reward =", sum(ucb_rewards))

```

UCB Policy: Total Reward = 131

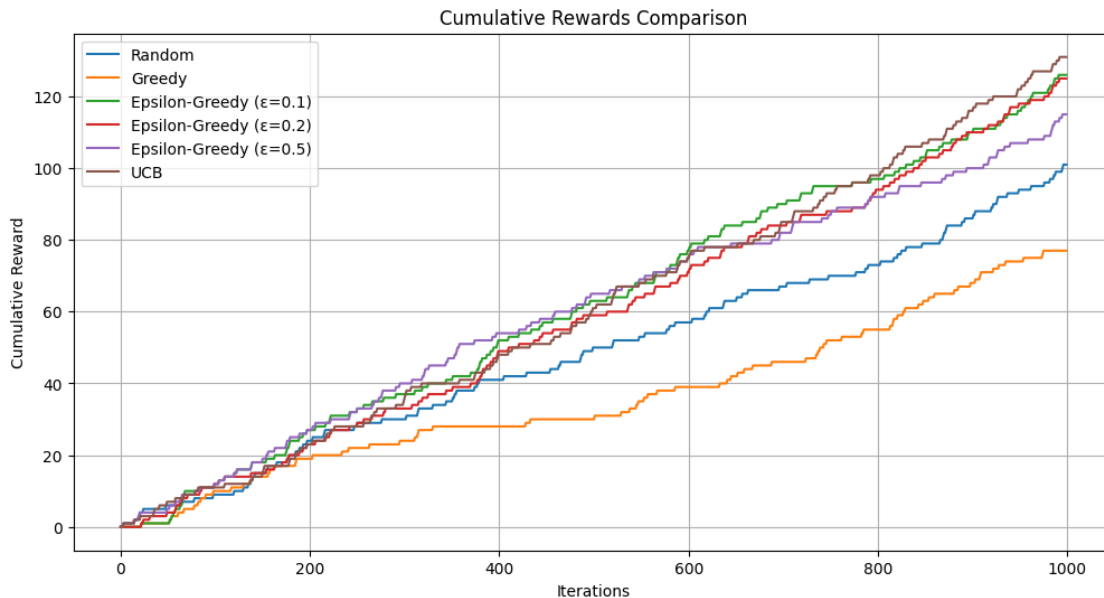
3.1 Plots

3.1.1 Cumulative Rewards

Why is this plot needed?

1. Tracks total success over time and shows how well a policy performs across trials.
2. Helps compare policies, steeper curves mean better performance.
3. Reveals learning speed and how quickly a policy finds the best treatment.
4. Highlights stability, smoother curves indicate consistent decision-making.

```
[15]: # Plot cumulative rewards
plt.figure(figsize=(12, 6))
plt.plot(np.cumsum(random_rewards), label='Random')
plt.plot(np.cumsum(greedy_rewards), label='Greedy')
for eps in epsilons:
    plt.plot(np.cumsum(epsilon_results[eps][0]), label=f'Epsilon-Greedy_{\u2192(={eps})}')
plt.plot(np.cumsum(ucb_rewards), label='UCB')
plt.title("Cumulative Rewards Comparison")
plt.xlabel("Iterations")
plt.ylabel("Cumulative Reward")
plt.legend()
plt.grid(True)
plt.show()
```



3.1.2 Arm Selection Frequency

Why is this plot needed?

1. Shows how often each treatment (arm) was chosen during the trial.
2. Helps understand exploration vs. exploitation in different strategies.
3. Reveals if the policy is stuck on a suboptimal arm.
4. Explains reward results by showing arm usage patterns.
5. Important for ethical insights in clinical decision-making.

```
[16]: # Plot arm selection frequency
labels = ['Arm 0', 'Arm 1', 'Arm 2', 'Arm 3']
x = np.arange(len(labels))
```

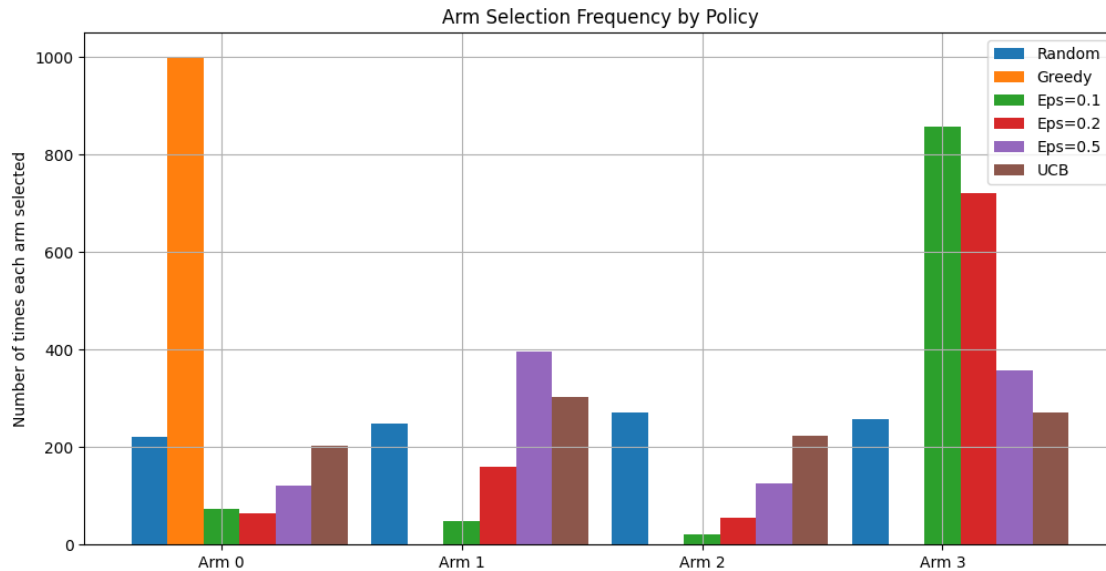
```

width = 0.15

fig, ax = plt.subplots(figsize=(12,6))
ax.bar(x - 2*width, random_counts, width, label='Random')
ax.bar(x - width, greedy_counts, width, label='Greedy')
for idx, eps in enumerate(epsilons):
    ax.bar(x + idx*width, epsilon_results[eps][1], width, label=f'Eps={eps}')
ax.bar(x + (idx+1)*width, ucb_counts, width, label='UCB')

ax.set_ylabel('Number of times each arm selected')
ax.set_title('Arm Selection Frequency by Policy')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
plt.grid(True)
plt.show()

```



3.2 Conclusion: Exploration vs. Exploitation

In this simulated clinical trial, we explored how different Multi-Armed Bandit strategies affect treatment selection outcomes. The **Random policy**, while fair, doesn't utilize any learning and hence performs poorly. The **Greedy policy**, although it exploits known information, lacks exploration and can get stuck in suboptimal arms.

The **-Greedy strategies** balance exploration and exploitation. $\epsilon = 0.1$ performed best in our setup, offering stability and adaptability. Larger ϵ (e.g., 0.5) resulted in more exploration but less focused learning.

The **UCB policy** consistently achieved high rewards. Its confidence-based exploration helps it

quickly identify the best arm and converge towards optimal decisions.

Overall, strategies that combine both **exploration (to discover better treatments)** and **exploitation (to use current knowledge)** yield the most reliable results in adaptive treatment selection — a key takeaway for ethical and efficient clinical trials.
